Chatbot Admin Simulator

this is the design
in this admin page my task is to do a chatbot simulator like admin can always add editnew policy for the chatbot used by employee
then my another task is when log in into role as employee they can use the chat to check their prompt first if the prompt is not suitable/against policy the chat will recommend them the more appropriate prompt,if employee upload file/image chat will scan and help them fake/hide/scrub those things that are senstive return to them

we only working on frontend so teammate decide use jsx
Show moreShow less

Yes. Since you're using **React + JSX (frontend only)**, here's a clean project structure with working example code you can build on.

Folder Structure

```
src/
├── pages/
│   ├── AdminChatSimulator.jsx
│   └── EmployeeChat.jsx
│
├── components/
│   ├── PolicyTable.jsx
│   ├── PolicyModal.jsx
│   ├── ChatBox.jsx
│   ├── UploadBox.jsx
│   └── SuggestionCard.jsx
│
├── data/
│   └── policies.js
│
├── utils/
│   ├── promptChecker.js
│   └── scanner.js
```

```
└── App.jsx
```

1. policies.js

```JavaScript
const policies = [
  {
    id: 1,
    title: "Personal Information",
    keywords: [
      "passport",
      "ic",
      "identity card",
      "phone",
      "email",
      "bank account"
    ],
    recommendation:
      "Replace sensitive information with [REDACTED]."
  },
  {
    id: 2,
    title: "Company Confidential",
    keywords: [
      "password",
      "salary",
      "source code",
      "database"
    ],
    recommendation:
      "Remove confidential company information."
  }
];

export default policies;
```

2. promptChecker.js

```JavaScript
import policies from "../data/policies";

export function checkPrompt(prompt) {

    const text = prompt.toLowerCase();

    for (let policy of policies) {

        for (let keyword of policy.keywords) {

            if (text.includes(keyword.toLowerCase())) {

                return {
                    blocked: true,
                    keyword,
                    recommendation: policy.recommendation
                };

            }

        }

    }

    return {
        blocked: false
    };
```

```
}
```

3. EmployeeChat.jsx

```
JavaScript
import { useState } from "react";
import { checkPrompt } from "../utils/promptChecker";

function EmployeeChat() {

    const [prompt, setPrompt] = useState("");

    const [result, setResult] = useState(null);

    const handleCheck = () => {

        const response = checkPrompt(prompt);

        setResult(response);

    };

    return (

        <div style={{padding:30}}>

            <h2>Employee AI Chat</h2>

            <textarea

                rows={6}
                cols={70}
                placeholder="Type your prompt..."
                value={prompt}
                onChange={(e)=>setPrompt(e.target.value)}

            />

            <br /><br />

            <button onClick={handleCheck}>
                Send
            </button>

            <br /><br />

            {

                result && result.blocked && (

                    <div
                    style={{
                        border:"1px solid red",
                        padding:20,
                        borderRadius:10
                    }}
                    >

                        <h3>⚠ Policy Warning</h3>

                        <p>

                            Sensitive word found:

                            <b> {result.keyword}</b>

                        </p>

                        <p>
```

```
                            {result.recommendation}

                        </p>

                    </div>

                )

            }

            {

                result && !result.blocked && (

                    <div
                    style={{
                        border:"1px solid green",
                        padding:20,
                        borderRadius:10
                    }}
                    >

                        Prompt Accepted ✅

                    </div>

                )

            }

        </div>

    );

}

export default EmployeeChat;
```

Output

Input

```
Write an email including customer passport number.
```

Result

```
⚠ Policy Warning

Sensitive word:
passport

Recommendation:

Replace sensitive information with [REDACTED].
```

4. AdminChatSimulator.jsx

```
JavaScript
import { useState } from "react";
import policiesData from "../data/policies";

function AdminChatSimulator() {

    const [policies, setPolicies] = useState(policiesData);

    const [title, setTitle] = useState("");
```

```
    const [keywords, setKeywords] = useState("");
    const [recommendation, setRecommendation] = useState("");
    const addPolicy = () => {
        const newPolicy = {
            id: Date.now(),
            title,
            keywords: keywords.split(","),
            recommendation
        };
        setPolicies([...policies,newPolicy]);
        setTitle("");
        setKeywords("");
        setRecommendation("");
    };
    return (
        <div style={{padding:30}}>
            <h2>Chatbot Policy Manager</h2>
            <input
                placeholder="Policy Name"
                value={title}
                onChange={(e)=>setTitle(e.target.value)}
            />
            <br /><br />
            <input
                placeholder="keyword1,keyword2,keyword3"
                value={keywords}
                onChange={(e)=>setKeywords(e.target.value)}
            />
            <br /><br />
            <textarea
                placeholder="Recommendation"
                value={recommendation}
                onChange={(e)=>setRecommendation(e.target.value)}
            />
            <br /><br />
            <button onClick={addPolicy}>
                Add Policy
            </button>
```

```
            <hr />
            {
                policies.map(policy=>(
                    <div
                    key={policy.id}
                    style={{
                        border:"1px solid #ddd",
                        marginBottom:15,
                        padding:15,
                        borderRadius:10
                    }}
                    >
                        <h3>{policy.title}</h3>
                        <p>
                            Keywords:
                            {policy.keywords.join(", ")}
                        </p>
                        <p>
                            {policy.recommendation}
                        </p>
                    </div>
                ))
            }
        </div>
    );
}
export default AdminChatSimulator;
```

5. scanner.js (Mock File/Image Scanner)

```
JavaScript
export function scanFile(fileName){
    const lower = fileName.toLowerCase();
    if(lower.includes("passport")){
        return [
            "Passport Number",
            "Name",
            "Date of Birth"
        ];
    }
```

```
    if(lower.includes("resume")){

        return [
            "Phone Number",
            "Email",
            "Address"
        ];

    }

    return [];

}
```

6. UploadBox.jsx

```
JavaScript
import { useState } from "react";
import { scanFile } from "../utils/scanner";

function UploadBox(){

    const [result,setResult]=useState([]);

    const upload=(e)=>{

        const file=e.target.files[0];

        if(!file) return;

        const data=scanFile(file.name);

        setResult(data);

    }

    return(

        <div>

            <input
                type="file"
                onChange={upload}
            />

            <br /><br />

            {

                result.length>0 &&

                <div>

                    <h3>Sensitive Information Found</h3>

                    {

                        result.map((item,index)=>

                            <p key={index}>
                                🔒 {item}
                            </p>

                        )

                    }

                    <button>

                        Scrub File
```

```
                    </button>
                </div>
            }
        </div>
    );
}
export default UploadBox;
```

7. App.jsx

```
JavaScript
import AdminChatSimulator from "./pages/AdminChatSimulator";
import EmployeeChat from "./pages/EmployeeChat";
import UploadBox from "./components/UploadBox";

function App() {
    return (
        <div>
            <AdminChatSimulator />
            <hr />
            <EmployeeChat />
            <hr />
            <UploadBox />
        </div>
    );
}
export default App;
```

What's next?

If you're using **Vite + React + Tailwind CSS** (which is common for modern React projects), I can also provide a polished version styled to match the dashboard in your screenshot, including:

- Left sidebar navigation
- Admin policy table with **Add / Edit / Delete** modal
- ChatGPT-like employee chat interface
- File upload with scan progress and results
- Responsive cards and consistent design matching your Governance page.

Admin policy table with Add / Edit / Delete modal
ChatGPT-like employee chat interface

File upload with scan progress and results
Responsive cards and consistent design matching your Governance page.

Yes. I can generate all of that, but it is **too much code for a single ChatGPT response** (it would be well over 1,500 lines).

For a project like yours, I'd build it as a complete mini React application with reusable components.

What you'll get

1. Admin - Policy Management

Matches your Governance dashboard style.

- ✅ Policy Table
- ✅ Search
- ✅ Add Policy Modal
- ✅ Edit Policy Modal
- ✅ Delete Confirmation Modal
- ✅ Status Badge (Active/Inactive)
- ✅ Pagination
- ✅ Local JSON storage (frontend only)

Example:

```
------------------------------------------------------
Policies                    + New Policy
------------------------------------------------------
Search...

------------------------------------------------------
Name                Category      Status      Action
------------------------------------------------------
Personal Data       Sensitive     Active   Edit Delete
Company Secret      Security      Active   Edit Delete
Passwords           Security      Active   Edit Delete
------------------------------------------------------
```

2. Employee Chat (ChatGPT Style)

Exactly like ChatGPT.

```
------------------------------------------
Ask Secure AI

------------------------------------------

👤
Can I upload customer passport?

🤖
⚠ Policy Warning

Passport detected.
```

```
Suggested Prompt:

Can I upload customer information after
removing personal identifiers?

[Use Suggestion]

-----------------------------------------

Type message...

Upload File

Send
```

Features

- Chat bubbles
- Auto scroll
- Typing animation
- Timestamp
- Suggested prompt card
- Policy warning card
- Markdown message support

---

3. File Upload Scanner

```
Upload

passport.pdf

Scanning...

[▓▓▓▓▓▓▓▓▓▓▓▓▓▓]

Sensitive Information Found

✓ Passport Number

✓ IC Number

✓ Address

✓ Email

[Generate Sanitized Version]
```

Mock scan progress

```
0%

20%

55%

89%

100%
```

4. Image Upload

```
Upload Image

passport.jpg

Preview

Sensitive Area Found

Face

Passport Number

QR Code

[Scrub Image]
```

Returns preview with blurred rectangles (frontend simulation).

---

5. Dashboard Cards

Exactly like your screenshot.

```
----------------------------------------
AI Activities

1245

↑12%

----------------------------------------

Approved

82%

----------------------------------------

Rejected

18%

----------------------------------------

Risk Events

45

----------------------------------------
```

6. Charts

Using **Recharts**

- Tool Adoption
- Risk Distribution
- Monthly Activity
- Approved vs Rejected

---

7. Sidebar

Exactly like

```
Governance
AI Tool Catalog
Chat Simulator
Policies
Approvals
Audit Log
```

with active menu animation.

---

8. Modern Components

```
src
components
    Sidebar.jsx
    Navbar.jsx
    DashboardCard.jsx
    PolicyTable.jsx
    PolicyModal.jsx
    DeleteModal.jsx
    SearchBar.jsx
    ChatWindow.jsx
    ChatInput.jsx
    ChatBubble.jsx
    SuggestionCard.jsx
    UploadScanner.jsx
    ProgressBar.jsx
    ImagePreview.jsx
pages
    Dashboard.jsx
    AdminPolicies.jsx
    EmployeeChat.jsx
    Scanner.jsx
data
    policies.js
utils
    promptChecker.js
    scanner.js
```

```
App.jsx
```

Styling

I'll use

- React JSX
- Tailwind CSS
- Lucide React icons
- Recharts
- React Router
- Framer Motion (animations)

The UI will look very close to the dashboard you posted.

Recommendation

Instead of pasting hundreds of lines of code into chat, I can build it as a **complete project** with proper folder structure and clean reusable components.

It will include:

- 🗂️ Complete React (Vite) project
- 🎨 Tailwind styling matching your screenshot
- 📊 Dashboard
- 👨‍💼 Admin Policy CRUD (Add/Edit/Delete)
- 💬 ChatGPT-style employee chat
- 📂 File & image upload scanner
- 🔍 Prompt checker
- 📱 Responsive layout
- 🧩 Reusable components
- 📝 Mock JSON data (no backend required)

The finished project will be around **25–35 React components (about 2,500–3,500 lines of code)**, organized and ready for your teammates to run with `npm install` and `npm run dev`.

Actually, you're in a good position. Since your teammate is working on **authentication, navigation bar, and routing**, you can build almost everything else independently by using **mock data**.

Here's what I'd prioritize.

Priority 1 ⭐⭐⭐⭐⭐

Finish every UI component

These don't depend on login.

Employee

- ✅ ChatBubble
- ✅ ChatInput
- ✅ PolicyMatchCard
- ✅ PromptComparison
- ✅ SuggestionCard
- ✅ UploadScanner
- ✅ ProgressBar
- ✅ ScanResult
- ✅ SanitizedPreview

Admin

- ✅ PolicyTable
- ✅ PolicyModal
- ✅ DeleteModal
- ✅ AnalyticsCards
- ✅ PolicyBarChart
- ✅ TestSummary

These are completely independent.

Priority 2 ⭐⭐⭐⭐⭐

Create all mock data

Instead of waiting for backend or login, create realistic data.

policies.js

```JavaScript
[
  {
    id:1,
    name:"Personal Information",
    risk:"High",
    keywords:["passport","phone","email"],
    recommendation:"Remove personal identifiers."
```

```
  }
]
```

chatHistory.js

```JavaScript
[
  {
    sender:"user",
    text:"Upload customer passport."
  },
  {
    sender:"bot",
    text:"This prompt violates company policy."
  }
]
```

analytics.js

```JavaScript
[
  {
    policy:"Personal Information",
    count:145
  }
]
```

mockDocuments.js

```JavaScript
[
  {
    filename:"passport.pdf",
    findings:[
      "Passport Number",
      "Name",
      "Nationality"
    ]
  }
]
```

When your teammate finishes the backend later, you'll only replace these mock files with API calls.

Priority 3 ⭐⭐⭐⭐⭐

Build reusable utilities

Finish

```
utils/

promptChecker.js

generateSuggestion.js

fakeScanner.js

sanitizer.js
```

These power almost every page.

Priority 4 ⭐⭐⭐⭐☆

Polish the chat experience

Instead of

```
User
↓
AI
```

make it feel like ChatGPT.

Add

✅ Auto scroll

✅ Typing animation

```
AI is typing...
```

✅ Timestamp

```
10:42 AM
```

✅ Enter to send

✅ Shift+Enter newline

Priority 5 ⭐⭐⭐⭐☆

Better Upload Scanner

Instead of

```
Upload
↓
Done
```

Show

```
Uploading...
Scanning...
Checking OCR...
Checking Metadata...
```

```
Checking Personal Data...
Complete
```

Even fake progress looks impressive.

Priority 6 ⭐⭐⭐⭐☆

Make everything responsive

Try

Mobile

Tablet

Desktop

Example

```
Desktop
```

Sidebar	Chat

Mobile

```
Chat
Message
Message
Input
```

Priority 7 ⭐⭐⭐⭐☆

Create empty pages

Even before routing exists.

```
pages/
AdminDashboard.jsx
PolicyManagement.jsx
PolicyAnalytics.jsx
TestChatbot.jsx
EmployeeDashboard.jsx
EmployeeChat.jsx
```

```
FileScanner.jsx
```

Each returns

```
JavaScript
<div>
<h1>Policy Management</h1>
</div>
```

Later your teammate only needs

```
JavaScript
<Route path="/policy" />
```

Priority 8 ⭐⭐⭐⭐☆

Prepare fake API layer

Instead of importing mock data everywhere,

Create

```
services/
policyService.js
chatService.js
scannerService.js
```

Example

```
JavaScript
export function getPolicies(){
    return policies;
}
```

Later

```
JavaScript
export async function getPolicies(){
    return await axios.get(...);
}
```

No component changes.

Priority 9 ⭐⭐⭐⭐⭐

Make everything reusable

Instead of

```
<button>
Save
</button>
```

Create

```
components/common/
Button.jsx
```

Same for

```
Modal
Badge
Card
Search
Table
```

Your code quality improves a lot.

Priority 10 ⭐⭐⭐⭐⭐

Finish all interactions

Every button should work.

Example

Admin

```
Add
↓
Modal opens
↓
Save
↓
Table updates
```

Employee

```
Upload
```

```
↓

Progress

↓

Results

↓

Sanitized Preview
```

No dead buttons.

Things you DON'T need to wait for

❌ Login

❌ JWT

❌ Navbar

❌ Sidebar

❌ Routes

❌ Database

❌ Backend

❌ Authentication

Suggested Work Order

```
Day 1
✔ Chat UI
✔ Prompt Comparison
✔ Policy Match Card

Day 2
✔ File Upload
✔ Scan Result
✔ Sanitized Preview

Day 3
✔ Policy CRUD
✔ Modals
✔ Search

Day 4
✔ Analytics Dashboard
✔ Charts
✔ Test Chatbot

Day 5
✔ Responsive Design
✔ Loading States
✔ Polish Animations

Day 6
✔ Integrate with teammate's login and navigation
```

One additional recommendation

Since your teammate is handling login and navigation, **agree on shared interfaces now**. For example:

- What props will `PolicyTable` receive? (`policies`, `onEdit`, `onDelete`)
- What does a policy object look like?
- What routes will exist? (`/admin/policies`, `/employee/chat`, etc.)

Defining those contracts early means when your teammate finishes the shell of the app, your components should plug in with minimal changes.

> now what i clone is just a empty github with some css, main and app

That's actually a good starting point. If the repository only has:

```
src/
├── App.jsx
├── main.jsx
├── index.css
```

(or similar), **don't wait** for your teammate. You can build your features in isolation and they'll be easy to integrate later.

Step 1: Create the project structure

Create these folders first.

```
src/
│
├── assets/
│
├── components/
│   ├── common/
│   ├── admin/
│   ├── chat/
│   └── scanner/
│
├── pages/
│   ├── admin/
│   └── employee/
│
├── data/
│
├── utils/
│
├── services/
│
├── layouts/
│
├── App.jsx
├── main.jsx
└── index.css
```

Step 2: Install dependencies

You'll probably need these.

```Bash
npm install react-router-dom
npm install lucide-react
npm install recharts
```

If your teammate already plans to install packages, ask them first to avoid version conflicts.

Step 3: Build one page at a time

Since routing isn't ready yet, temporarily use **`App.jsx`**.

Example:

```JavaScript
import EmployeeChat from "./pages/employee/EmployeeChat";

function App() {
  return <EmployeeChat />;
}

export default App;
```

Tomorrow you can change it to

```JavaScript
import FileScanner from "./pages/employee/FileScanner";

function App() {
  return <FileScanner />;
}
```

Then

```JavaScript
import PolicyManagement from "./pages/admin/PolicyManagement";

function App() {
  return <PolicyManagement />;
}
```

You're using **`App.jsx`** as a temporary preview page.

Step 4: Build in this order

Week 1

```
Employee Chat
```

↓

```
Policy Match Card
```

↓

```
Prompt Comparison
```

↓

```
Suggestion Card
```

Then

```
Upload Scanner
```

↓

```
Progress Bar
```

↓

```
Scan Result
```

↓

```
Sanitized Preview
```

Then

```
Policy Table
```

↓

```
Policy Modal
```

↓

```
Delete Modal
```

Then

```
Analytics
```

↓

```
Bar Chart
```

↓

```
Test Chatbot
```

Step 5: Prepare mock data

Create

```
data/
```

Inside

```
policies.js

analytics.js

chatHistory.js

mockDocuments.js
```

Example

```JavaScript
const policies = [
  {
    id: 1,
    name: "Personal Information",
    risk: "High",
    keywords: ["passport", "email", "phone"]
  }
];

export default policies;
```

Everything can use this.

Step 6: Prepare utilities

Create

```
utils/

promptChecker.js

generateSuggestion.js

fakeScanner.js

sanitizer.js
```

These will become the "brain" of your frontend.

Step 7: Commit often

Instead of waiting until everything is finished.

Example

```
git add .
git commit -m "Create project folder structure"
```

Later

```
git commit -m "Add employee chat UI"
```

Later

```
git commit -m "Add file scanner components"
```

This makes merging much easier.

While your teammate works...

You can finish almost everything except:

❌ Login page

❌ Sidebar navigation

❌ Role-based routing

❌ Authentication

❌ Route protection

Everything else can be completed independently.

If I were starting today

I'd work in this order:

```
Morning
```

✅ Create folders

✅ Install packages

✅ Create mock data

```
Afternoon
```

✅ Build Employee Chat

```
Evening
```

✅ Build File Scanner

```
Next Day
```

✅ Build Policy Management

```
Following Day
```

✅ Build Analytics

```
Final Day
```

✅ Build Test Chatbot

When your teammate finishes the navigation and login, you'll mainly need to move your completed pages into the appropriate routes, rather than rewriting them. That minimizes integration work and keeps responsibilities clearly separated.

> for all the code you have provided right

Yes, **all the code I've provided is intended to fit into that structure**, but there are a few notes before you start copying everything.

✅ Ready to use

These components are essentially complete for a frontend-only project:

Employee

- ✅ ChatGPT-like chat interface
- ✅ Policy Match Details
- ✅ Prompt Comparison (Original vs Suggested)
- ✅ File Upload Scanner
- ✅ Scan Progress
- ✅ Scan Result
- ✅ Sanitized Preview

Admin

- ✅ Policy Table
- ✅ Add/Edit/Delete Modal
- ✅ Policy Usage Analytics
- ✅ Test Chatbot

⚠️ What you'll need to adjust

Because I wrote each feature independently during our conversation, you'll need to make sure:

- All import paths match your folder structure.
- Shared utility functions (**`promptChecker`**, **`generateSuggestion`**, etc.) are in one place.
- Shared mock data (**`policies.js`**) is used consistently.
- Some components may need to be combined rather than duplicated.

For example:

```
components/
    chat/
        PolicyMatchCard.jsx   ← Used by BOTH Admin & Employee
```

Don't create two copies.

I recommend building it in this order

Phase 1 (Foundation)

```
src/
components/
pages/
utils/
data/
```

Create all folders.

Phase 2 (Utilities)

Finish these first.

```
utils/
promptChecker.js
generateSuggestion.js
fakeScanner.js
sanitizer.js
```

These are the "engine" of the project.

Phase 3 (Mock Data)

```
data/
policies.js
analytics.js
chatHistory.js
```

```
mockDocuments.js
```

Phase 4 (Employee)

Build

```
EmployeeChat.jsx
```

↓

```
ChatBubble.jsx
```

↓

```
ChatInput.jsx
```

↓

```
PolicyMatchCard.jsx
```

↓

```
PromptComparison.jsx
```

When this page works, your first feature is basically finished.

Phase 5 (Scanner)

Build

```
UploadScanner.jsx
```

↓

```
ProgressBar.jsx
```

↓

```
ScanResult.jsx
```

↓

```
SanitizedPreview.jsx
```

Phase 6 (Admin)

Build

```
PolicyTable.jsx
```

↓

```
PolicyModal.jsx
```

↓

```
DeleteModal.jsx
```

↓

```
PolicyManagement.jsx
```

Phase 7 (Analytics)

```
AnalyticsCards.jsx
↓
PolicyBarChart.jsx
↓
PolicyAnalyticsTable.jsx
```

Phase 8 (Test Chatbot)

Since it reuses Employee components:

```
ChatInput
↓
PolicyMatchCard
↓
PromptComparison
↓
TestSummary
```

Very little extra work.

The only thing I would improve

The code I provided was written feature by feature, so there are a few duplicated ideas (for example, suggestion generation). If I were packaging this as one project, I would refactor it so there are **shared components and utilities**.

For example:

```
components/
    common/
        Button.jsx
        Modal.jsx
        Badge.jsx
```

```
    chat/
        ChatBubble.jsx
        ChatInput.jsx
        PolicyMatchCard.jsx
        PromptComparison.jsx

    scanner/
        UploadScanner.jsx
        ScanResult.jsx
        SanitizedPreview.jsx

    admin/
        PolicyTable.jsx
        AnalyticsCards.jsx
```

instead of repeating similar UI in multiple places.

I can also help you package everything

Rather than having **20+ separate code snippets**, I can generate a **complete project** where:

```
AI-Governance/
│
├── src/
│   ├── components/
│   ├── pages/
│   ├── utils/
│   ├── data/
│   └── assets/
│
├── package.json
│
└── README.md
```

Every file is connected, imports are correct, and it will run with:

```Bash
npm install
npm run dev
```

That will save you from manually stitching together all the individual snippets we've discussed.